

Linux高级调试技巧：GDB反向调试，让程序逆序执行，调试代码原来这么简单！

西北望msm66g9f 2024-05-13 发布于山东 | 123阅读 | 1转藏

转藏

全屏

朗读

分享



反向调试(Remote Debugging)有个很拉风的别名——Time Travel Debugging。是的，你没看错，就是时间旅行调试！光看这么名字，就知道它有多牛X了吧！为了尽量讲清楚，本文较长，请耐心等待，不会让你失望！

引子

在调试程序的时候，你是否遇到下面这些情况：

- 在GDB中跟踪程序执行，段错误发生时，bt查看调用栈，却全是问号(当然编译时已经加了-g)，无从下手。
- 在调试逻辑比较复杂的程序，程序执行到某个点时，突然发现一些异常，比如一个变量值不对，想查看一下这个变量在前面某个点时的值是多少，于是不得不中断调试，重新执行程序，从头开始调试。
- 调试多线程程序时，经常会由于设置断点等原因，改变程序正常运行时线程间的时序逻辑，导致问题很难复现，不得不反复进行复现调试。
- 定位到问题发生在某个函数中，可这个函数却被多个地方甚至多个线程多次调用，在这个函数设置断点，导致程序经常被中断执行，而这些被触发的断点，并不是自己想要的。

本文将采用实例的方式，详细介绍一种针对上面这些情况简单有效的调试方法 - 反向调试

在真正介绍反向调试之前，先看一个例子，来展示下反向调试究竟有多大威力！

反向调试的威力——解决调用者全是问号的难题

本文的测试代码，在正常运行时会触发Segmentation fault，并且调用栈显示全是问号(编译时当然已经添加了-g选项！)：

```
Program stopped.      // 触发Segmentation fault, 程序停下来了
0x000056f1000056dc in ?? ()
(gdb) bt              // 调用栈全是问号
#0  0x000056f1000056dc in ?? ()
#1  0x0000572100005708 in ?? ()
#2  0x000057590000573c in ?? ()
#3  0x0000579900005778 in ?? ()
#4  0x000057e1000057bc in ?? ()
#5  0x0000583100005808 in ?? ()
#6  0x000058890000585c in ?? ()
#7  0x00000000000401020 in ?? ()
#8  0x00007fffffffe440 in ?? ()
#9  0x0000000000000000 in ?? ()
(gdb)
```

使用本文介绍的逆向调试的方法，经过简单的调试，可以在不修改代码，不重新编译的前提下，让调用者显示正常，轻松找到Root Cause！

```
(gdb) bt              // 通过逆向调试，调用栈显示正常，直接定位到Root Cause
#0  0x00000000000401142 in bar (array=0x7fffffffe330, len=24) at corrupt.c:4
#1  0x0000000000040117b in foo (array=0x7fffffffe330, len=24) at corrupt.c:10
#2  0x000000000004011ad in main () at corrupt.c:16
(gdb)
```

是不是很神奇？这只是反向调试威力的冰山一角而已！接下来，开始正式介绍反向调试！

什么是反向调试？

反向调试是一种高级调试技术，可以让程序已经执行了一段时间后，回退到过去的状态并重新执行。这意味着你可以回到程序执行中的任何点，查看变量的值、堆栈跟踪以及程序执行路径。反向调试可以让我们快速、准确地定位出程序中的错误或异常的根本原因。

简单来说，**就是一种可以让程序逻辑逆序执行的调试技术。通过它，你可以随时中断程序正常执行，然后让它逆序执行，让程序回到过去，可以查看任意时间点的任意信息。**

先通过一个简单的例子，直观感受一下。

源码如图：



```
1 int main()  
2 {  
3     int a = 0;  
4     a = 1;  
5     a = 2;  
6     a = 3;  
7     return 0;  
8 }
```

在GDB中启动程序，并停在程序入口处，并在第7行设置断点，以便让程序停下来，让我们进行反向调试

```
root@ubuntu:ReverseDebugging# gdb test  
Reading symbols from test...  
(gdb) start  
Temporary breakpoint 1 at 0x40111d: file test.c, line 3.  
Starting program: /opt/data/workspace/articles/gdb/articles/ReverseDebugging/test  
  
Temporary breakpoint 1, main () at test.c:3  
3         int a = 0;  
(gdb) b 7  
Breakpoint 2 at 0x401139: file test.c, line 7.
```

执行record命令，开始记录程序执行轨迹

```
(gdb) record
```

执行c(continue)命令，让程序正常执行，然后在第7行触发断点，停下来

```
(gdb) c  
Continuing.  
  
Breakpoint 2, main () at test.c:7  
7         return 0;
```

此时，查看a的值，a = 3

```
(gdb) p a  
$1 = 3
```

执行reverse-next命令，让程序逆向执行一行代码。

```
(gdb) reverse-next  
6         a = 3;
```

准确地说，这条命令的作用是让程序回退一行代码的执行效果。也就是回退到刚执行完第5行代码，尚未执行第6行代码时的状态。此时查看a的值，不出意外的话，a的值应该是2，我们来验证一下

```
(gdb) p a
$2 = 2
```

果然，第6行代码的执行效果已经被完全回退了。我们再执行一条reverse-next命令，此时，应该是回退第5行代码的执行效果。

```
(gdb) reverse-next
5          a = 2;
```

此时，程序的状态已经被回退到刚执行完第4行代码，尚未执行第5行代码时的状态，此时，a应该是1，验证一下：

```
(gdb) p a
$3 = 1
```

然后，回退第4行代码的执行效果，让程序状态恢复到刚执行完第3行代码，尚未执行第4行代码时的状态：

```
(gdb) reverse-next
4          a = 1;
(gdb) p a
$4 = 0
```

到这里，大家应该已经明白了吧，**reverse-next**的作用是让程序恢复到上一行代码刚执行完时的状态。当然，我们仍然可以让程序以正常顺序执行下去：

```
(gdb) c
Continuing.

No more reverse-execution history.
main () at test.c:7
7          return 0;
(gdb) p a
$5 = 3
(gdb)
```

再看一下整个调试过程，加深一下理解：

```
1 root@ubuntu:ReverseDebugging# gdb test
2 Reading symbols from test...
3 (gdb) start
4 Temporary breakpoint 1 at 0x40111d: file test.c, line 3.
5 Starting program:
  /opt/data/workspace/articles/gdb/articles/ReverseDebugging/test
6
7 Temporary breakpoint 1, main () at test.c:3
8 3      int a = 0;
9 (gdb) b 7          // 在第7行设置断点
10 Breakpoint 2 at 0x401139: file test.c, line 7.
11 (gdb) record      // 开始记录程序执行轨迹
12 (gdb) c          // 让程序继续正常执行
13 Continuing.
14
15 Breakpoint 2, main () at test.c:7  // 触发第7行断点
16 7      return 0;
17 (gdb) p a        // 此时, a = 3
18 $1 = 3
19 (gdb) reverse-next // 逆序执行一行代码
20 6      a = 3;
21 (gdb) p a        // 此时, a = 2
22 $2 = 2
23 (gdb) reverse-next // 逆序执行一行代码
24 5      a = 2;
25 (gdb) p a        // 此时, a = 1
26 $3 = 1
27 (gdb) reverse-next // 逆序执行一行代码
28 4      a = 1;
29 (gdb) p a        // 此时, a = 0
30 $4 = 0
31 (gdb) c          // 恢复程序正常执行
32 Continuing.
33
34 No more reverse-execution history.
35 main () at test.c:7  // 仍然会在第7行断点处停下
36 7      return 0;
37 (gdb) p a        // 此时, a = 3
38 $5 = 3
39 (gdb)
```

反向调试实现原理

反向调试技术的核心原理，简单来说，是在程序运行中，记录每一条指令对程序执行的所有状态变化，包括变量、寄存器、内存的数据变化等，并将这些信息存储在一个history文件中。当需要回溯到过去的状态时，调试器会按照相反的顺序逐条指令恢复这些状态，使得程序的执行状态回到已经被记录的任意时间点。

因此，反向调试还有一个雅称 - Time Travel Debugging

GDB反向调试用法

GDB作为Linux下的调试神器，很早就支持了反向调试的功能，常用的命令有：

```
root@ubuntu:ReverseDebugging# gdb
(gdb) reverse-
reverse-continue  reverse-finish  reverse-next      reverse-nexti
reverse-search    reverse-step   reverse-stepi
(gdb)
```

简单解释一下：

- reverse-next(**rn**): 参考next(n)，逆向执行一行代码，遇函数调用不进入
- reverse-nexti(rni): 参考nexti(ni)，逆向执行一条指令，与函数调用不进入
- reverse-step(rs): 参考step(s)，逆向执行一行代码，遇函数调用则进入
- reverse-stepi(rsi): 参考setpi(si)，逆向执行一条指令，与函数调用则进入
- reverse-continue(rc): 参考continue(c)，逆向继续执行
- reverse-finish: 参考finish，逆向执行，一直到函数入口处
- reverse-search(): 参考search，逆向搜索
- set exec-direction reverse: 设置程序逆向执行，执行完此命令后，所有常用命令如next, nexti, step, stepi, continue、finish等全部都变成逆向执行
- set exec-direction forward: 设置程序正向执行，这也是默认的设置

在绝大多数环境下，在使用这些反向调试命令之前，必须要先通过record命令让GDB把程序执行过程中的所有状态信息全部记录下来。

通常是在程序运行起来之后，先设置断点让程序停下来，然后输入record命令，开启状态信息记录，然后再继续执行。相关常用命令有：

- record: 记录程序执行过程中所有状态信息
- record stop: 停止记录状态信息
- record goto: 让程序跳转到指定的位置，如record goto start、record goto end、record goto n
- record save filename: 把程序执行历史状态信息保存到文件，默认名字是gdb_record.process_id
- record restore filename: 从历史记录文件中恢复状态信息
- show record full insn-number-max: 查看可以记录执行状态信息的最大指令个数，默认是200000
- set record full insn-number-max limit: 设置可以记录执行状态信息的最大指令个数
- set record full insn-number-max unlimited: 记录所有指令的执行状态信息

还有其他一些不常用的命令，不再赘述，感兴趣的童鞋可以查看GDB帮助文档。

下面我们来深入研究下，反向调试到底有什么用，该怎么去用。

反向调试有什么用

反向调试在定位这些类型的问题时会特别有用：

- 很难复现的BUG

- 逻辑非常复杂的问题
- 多线程并发问题
- 内存相关的问题，如内存踩踏，内存泄漏、重复释放等
- 调用栈显示全是问号的问题

下面，以调用栈显示全是问号的问题为例，演示一下反向调试的使用方法。

实例：Segmentation fault时调用栈全是问号

程序正常执行时，会发生Segmentation fault：

```
root@ubuntu:ReverseDebugging# ./corrupt
Segmentation fault (core dumped)
root@ubuntu:ReverseDebugging#
```

默认生产了core dump，用GDB直接调试core，直接bt：

```
root@ubuntu:ReverseDebugging# gdb --core=./core-corrupt-317563-1694865151
[New LWP 317563]
Core was generated by `./corrupt'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x000056f1000056dc in ?? ()
(gdb) bt
#0  0x000056f1000056dc in ?? ()
#1  0x0000572100005708 in ?? ()
#2  0x000057590000573c in ?? ()
#3  0x0000579900005778 in ?? ()
#4  0x000057e1000057bc in ?? ()
#5  0x0000583100005808 in ?? ()
#6  0x000058890000585c in ?? ()
#7  0x000000000000401020 in ?? ()
#8  0x00007ffcb4abcb20 in ?? ()
#9  0x0000000000000000 in ?? ()
(gdb)
```

居然全是问号！（注：编译时已经添加-g选项）

直接在GDB中运行试试看：


```
root@ubuntu:ReverseDebugging# gdb corrupt
Reading symbols from corrupt...
(gdb) r
Starting program:
/opt/data/workspace/articles/gdb/articles/ReverseDebugging/corrupt

Program received signal SIGSEGV, Segmentation fault.
0x000056f1000056dc in ?? ()
(gdb) bt
#0  0x000056f1000056dc in ?? ()
#1  0x0000572100005708 in ?? ()
#2  0x000057590000573c in ?? ()
#3  0x0000579900005778 in ?? ()
#4  0x000057e1000057bc in ?? ()
#5  0x0000583100005808 in ?? ()
#6  0x000058890000585c in ?? ()
#7  0x000000000000401020 in ?? ()
#8  0x00007fffffffef440 in ?? ()
#9  0x0000000000000000 in ?? ()
(gdb)
```

仍然全是问号！现在怎么办呢？

是不是有点慌了？淡定！咱们先来分析一下！

分析

既然bt显示全是问号，那我们就从这个现象着手进行分析。

我们知道bt (backtrace) 的作用是获取程序当前位置的函数调用关系。而函数调用关系，则是根据栈帧结构，从最深层次函数开始，逐层推导出来的。简单来说，就是逐层从栈中获取函数的返回地址，根据返回地址，从ELF的debug信息中找到对应的函数名、代码行号等信息。

(函数栈帧结构不是本文重点，篇幅所限，不再赘述，感兴趣的小伙伴可以加我微信CreCoding讨论)。

现在，bt打印出来全是问号，说明GDB无法获取到所需要的信息，也就是说，栈中的数据被破坏了！

那么思考一下，我们该如何定位究竟是哪把栈中数据给破坏了呢？

printf? No! 不解释!

直接单步调试吗？对应代码量不大，且逻辑简单的程序，这不失为一种可行的调试手段，但不够高效，往往需要反复猜测，反复重启程序调试才有可能找到真正的root cause。No!

直接数据断点？看起来确实是最简单有效，可问题是，往哪个地址设置断点呢？而且，如果程序正常运行过程中，会反复去修改这个地址里的数据呢？No！

反向调试？我们让程序正常运行，一直到触发segmentation fault。既然已经确定是栈数据被破坏，那么我们直接在栈上设置数据断点，然后让程序逆向执行，第一个触发数据断点的地方，肯定就是把这个错误的数​​据写到这个栈地址的代码，也就是破坏我们栈数据的地方！

Yes! Yes!

思路有了，Let's Go!

用反向调试进行定位

重新在GDB中运行程序，用**record**命令开始记录程序执行运行过程中的状态信息，然后让程序正常执行，直到发生segmentation fault停下来：

```
root@ubuntu:ReverseDebugging# gdb corrupt
Reading symbols from corrupt...
(gdb) start
Temporary breakpoint 1 at 0x4011a3: file corrupt.c, line 18.
Starting program:
/opt/data/workspace/articles/gdb/articles/ReverseDebugging/corrupt

Temporary breakpoint 1, main () at corrupt.c:18
18          foo(array, sizeof(array));
(gdb) record full
(gdb) c
Continuing.
Process record: failed to record execution log.

Program stopped.
0x000056f1000056dc in ?? ()
(gdb) bt
#0  0x000056f1000056dc in ?? ()
#1  0x0000572100005708 in ?? ()
#2  0x000057590000573c in ?? ()
#3  0x0000579900005778 in ?? ()
#4  0x000057e1000057bc in ?? ()
#5  0x0000583100005808 in ?? ()
#6  0x000058890000585c in ?? ()
#7  0x000000000000401020 in ?? ()
#8  0x00007fffffffef440 in ?? ()
#9  0x0000000000000000 in ?? ()
(gdb)
```

查看RSP寄存器，找到当前栈地址，然后在当前栈地址上设置数据断点：

```

(gdb) p $rsp
$1 = (void *) 0x7fffffffef360
(gdb) set can-use-hw-watchpoints 0
(gdb) watch *(long *)0x7fffffffef360
Watchpoint 2: *(long *)0x7fffffffef360
(gdb) i b

```

Num	Type	Disp	Enb	Address	What
2	watchpoint	keep	y		*(long *)0x7fffffffef360

```

(gdb)

```

我们直接在RSP地址处设置数据断点。需要解释一下的是，**我所用的GDB版本中，在进行反向调试时，硬件数据断点会失效，因此需要通过“set can-use-hw-watchpoints 0”命令强制GDB使用软件断点。**不知道最新的GDB版本中这个问题是否已经解决，感兴趣的童鞋可以去试一下。

然后，使用rc(reverse-continue)命令让程序开始逆向执行：

```

(gdb) reverse-continue
Continuing.

Watchpoint 2: *(long *)0x7fffffffef360

Old value = 95799245559560
New value = 140733193410312
0x0000000000401142 in bar (array=0x7fffffffef330, len=24) at corrupt.c:4
4          array[i] = 0x5678 + i * i;
(gdb)

```

可以看到，我们上一步在栈上设置的数据断点被成功触发了，程序停在了第4行，给数组元素赋值的地方，看一下这个元素的地址：

```

(gdb) p array
$2 = (int *) 0x7fffffffef330
(gdb) p i
$3 = 13
(gdb) p &array[i]
$4 = (int *) 0x7fffffffef364
(gdb)

```

确实是我们刚才设置的数据断点的地址。此时，再看下调用栈信息呢？

```
(gdb) bt
#0  0x0000000000401142 in bar (array=0x7fffffff330, len=24) at corrupt.c:4
#1  0x000000000040117b in foo (array=0x7fffffff330, len=24) at corrupt.c:10
#2  0x00000000004011ad in main () at corrupt.c:16
(gdb)
```

调用栈已经恢复正常！进一步证明，此时触发数据断点的这个地方，也就是第4行，就是破坏我们栈数据的罪魁祸首！

由于接下来都是常规调试，没有什么技术难点，篇幅所限，不再赘述。有疑问的童鞋可以加我微信CreCoding讨论！直接说结论：很容易分析出来，传入bar的数组长度超出了数组的大小，数组访问越界了，踩掉了栈中函数bar的返回地址，在return的时候返回到了一个非法地址，最终导致segmentation fault。

至此，BUG定位结束！

这段测试程序源码如下，建议亲自上手操作一下，加深理解：



```
void bar(int *array, int len)
{
    for(int i = 0; i < len; i++) {
        array[i] = 0x5678 + i * i;
    }
}

void foo(int *array, int len)
{
    bar(array, len);
}

int main(void)
{
    int array[6];
    foo(array, sizeof(array));
    return 0;
}
```

调试过程如下：


```

root@ubuntu:ReverseDebugging# gdb corrupt
Reading symbols from corrupt...
(gdb) start // start命令会启动被调试程序，并停在main()函数入口处
Temporary breakpoint 1 at 0x4011a3: file corrupt.c, line 16.
Starting program: /opt/data/workspace/articles/gdb/articles/ReverseDebugging/corrupt

Temporary breakpoint 1, main () at corrupt.c:16
16      foo(array, sizeof(array));
(gdb) record // 开始记录程序执行状态信息
(gdb) c // 程序继续执行
Continuing.
Process record: failed to record execution log.

Program stopped. // 触发Segmentation fault, 程序停下来了
0x000056f1000056dc in ?? ()
(gdb) bt // 调用栈全是问号
#0  0x000056f1000056dc in ?? ()
#1  0x0000572100005708 in ?? ()
#2  0x000057590000573c in ?? ()
#3  0x0000579900005778 in ?? ()
#4  0x000057e1000057bc in ?? ()
#5  0x0000583100005808 in ?? ()
#6  0x000058890000585c in ?? ()
#7  0x00000000000401020 in ?? ()
#8  0x00007fffffffe440 in ?? ()
#9  0x0000000000000000 in ?? ()
(gdb) p $rsp // 查看当前栈地址
$1 = (void *) 0x7fffffffe360
(gdb) set can-use-hw-watchpoints 0 // 硬件断点无法生效，强制GDB使用软件断点
(gdb) watch *(long *)0x7fffffffe360 // 设置数据断点
Watchpoint 2: *(long *)0x7fffffffe360
(gdb) reverse-continue // 开始逆向执行
Continuing.

Watchpoint 2: *(long *)0x7fffffffe360 // 触发数据断点

Old value = 95799245559560
New value = 140733193410312
0x00000000000401142 in bar (array=0x7fffffffe330, len=24) at corrupt.c:4
4      array[i] = 0x5678 + i * i; // 就是这一行破坏了栈上数据
(gdb) bt // 正序状态恢复到这里时，调用栈显示正常
#0  0x00000000000401142 in bar (array=0x7fffffffe330, len=24) at corrupt.c:4
#1  0x0000000000040117b in foo (array=0x7fffffffe330, len=24) at corrupt.c:10
#2  0x000000000004011ad in main () at corrupt.c:16
(gdb) p &array[i] // 查看相关变量信息
$2 = (int *) 0x7fffffffe364
(gdb) p i
$3 = 13
(gdb) p len
$4 = 24
(gdb) d
Delete all breakpoints? (y or n) y // 删除前面设置的数据断点，以便继续逆向执行
(gdb) reverse-finish // 继续逆向执行，一直到函数被调用的地方
Run back to call of #0 0x00000000000401142 in bar (array=0x7fffffffe330, len=24) at corrupt.c:4
0x00000000000401176 in foo (array=0x7fffffffe330, len=24) at corrupt.c:10
10      bar(array, len);
(gdb) reverse-finish // 继续逆向执行，一直到函数被调用的地方
Run back to call of #0 0x00000000000401176 in foo (array=0x7fffffffe330, len=24) at corrupt.c:10
0x000000000004011a8 in main () at corrupt.c:16
16      foo(array, sizeof(array));
(gdb) list // 查看源码
11      }

```



```
11     },
12
13     int main(void)
14     {
15         int array[6];           // 数组元素个数为6
16         foo(array, sizeof(array)); // 传入元素个数错误, 这就是Root Cause!
17         return 0;
18     }
19
20 (gdb)
```

反向调试的缺点

前面介绍过, 反向调试的实现原理是, 调试器在程序执行每一条指令时, 把这条指令所产生的效果记录下来, 比如修改了寄存器的值、修改了内存中的值、跳转到另外一个地址等等, 然后反向执行的时候, 逐条把这些指令产生的效果还原回来, 以此达到恢复程序执行状态的效果。

由此可见, 这势必会在程序执行过程中产生不小的额外开销, 对程序的性能产生比较大的影响。

不过, 对于一些对性能不敏感的程序, 或者规模不大的程序, 可以不考虑这个问题。而对于一下规模较大且对性能非常敏感的程序, 其实没有必要在程序执行一开始就记录执行轨迹状态信息, 可以考虑分段记录, 或者只记录自己感兴趣的程序段的状态信息。

思考题

1. 逆向执行时, 所有过去的状态信息都可以恢复吗?
2. 我们前面通过反向调试定位调用栈全是问号的这个问题, 在segmentation fault触发之后, 我们直接在RSP地址处设置了数据断点, 然后很轻松就定位到了root cause。思考一下, 这种方式在任何时候都是有效的吗? 有哪些情况下, 是不能直接在RSP地址处设置数据断点的, 或者说在RSP地址处设置断点是无法直接定位到破坏栈数据的那一行代码的呢? 如果出现这种情况, 该怎么办呢?